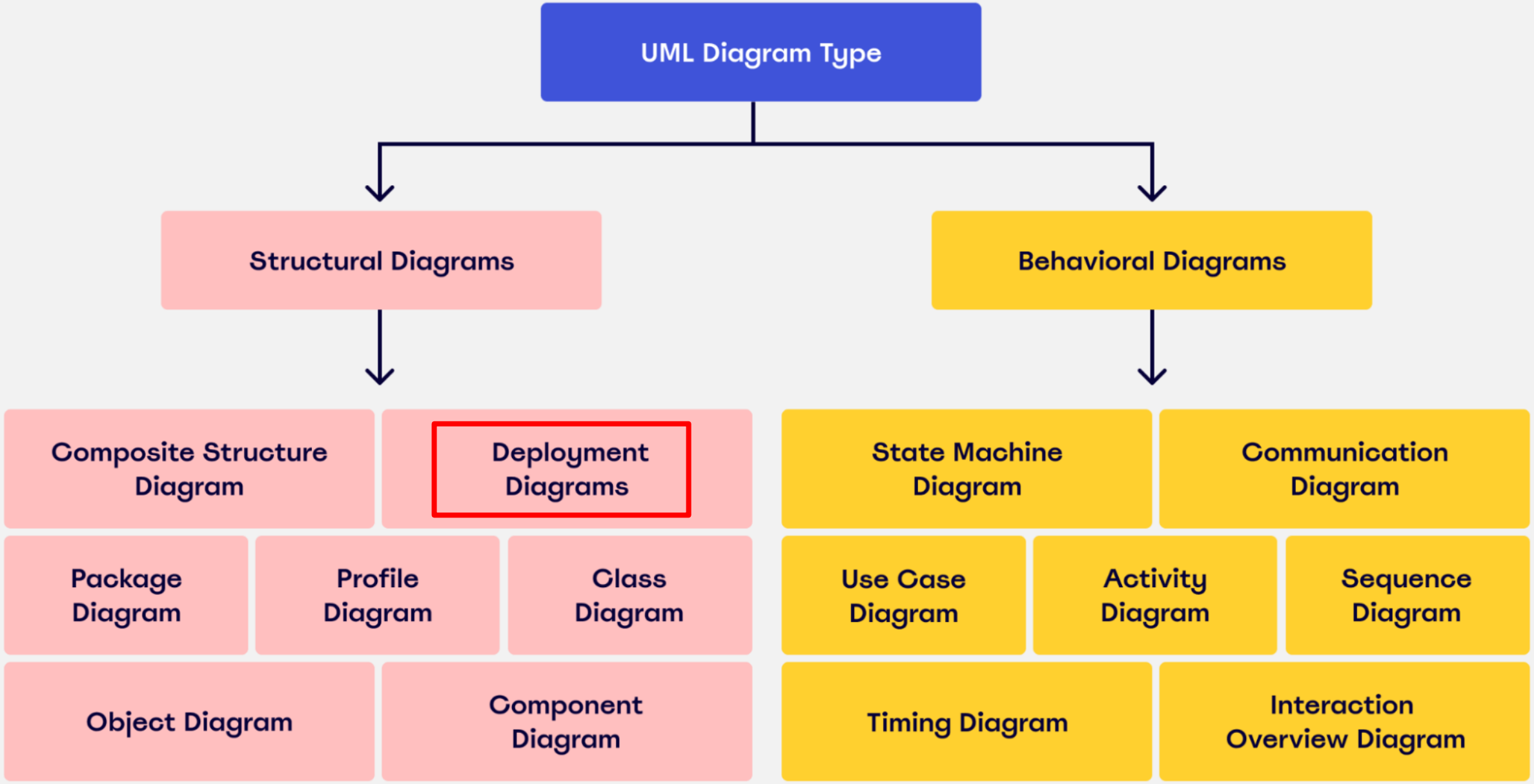
Object Oriented Analysis and Design

Deployment and Component Diagram


UML Diagram Type
Structural Diagrams
Behavioral Diagrams
Composite Structure Diagram
Deployment Diagrams
Package Diagram
Profile Diagram
Class Diagram
Object Diagram
Component Diagram
State Machine Diagram
Communication Diagram
Use Case Diagram
Activity Diagram
Sequence Diagram
Timing Diagram
Interaction Overview Diagram

Why it matters?

- A software release is more than just lines of code bundled together. It involves
  - Separate Libraries
  - An executable
  - An installer
  - Configuration files
  - Other different pieces



Why These Diagrams Matter?

- Real-world importance
- Modern software doesn’t run in isolation. Understanding deployment is critical for:
  - DevOps & Cloud: AWS, Azure, Kubernetes deployments
  - Scalability: Designing systems that grow with demand
  - Cost Optimization: Efficient resource allocation
  - Team Communication: Clear architecture documentation
  - Troubleshooting: Understanding where components live

Think of it as a blueprint for your software infrastructure

What are Deployment Diagrams?

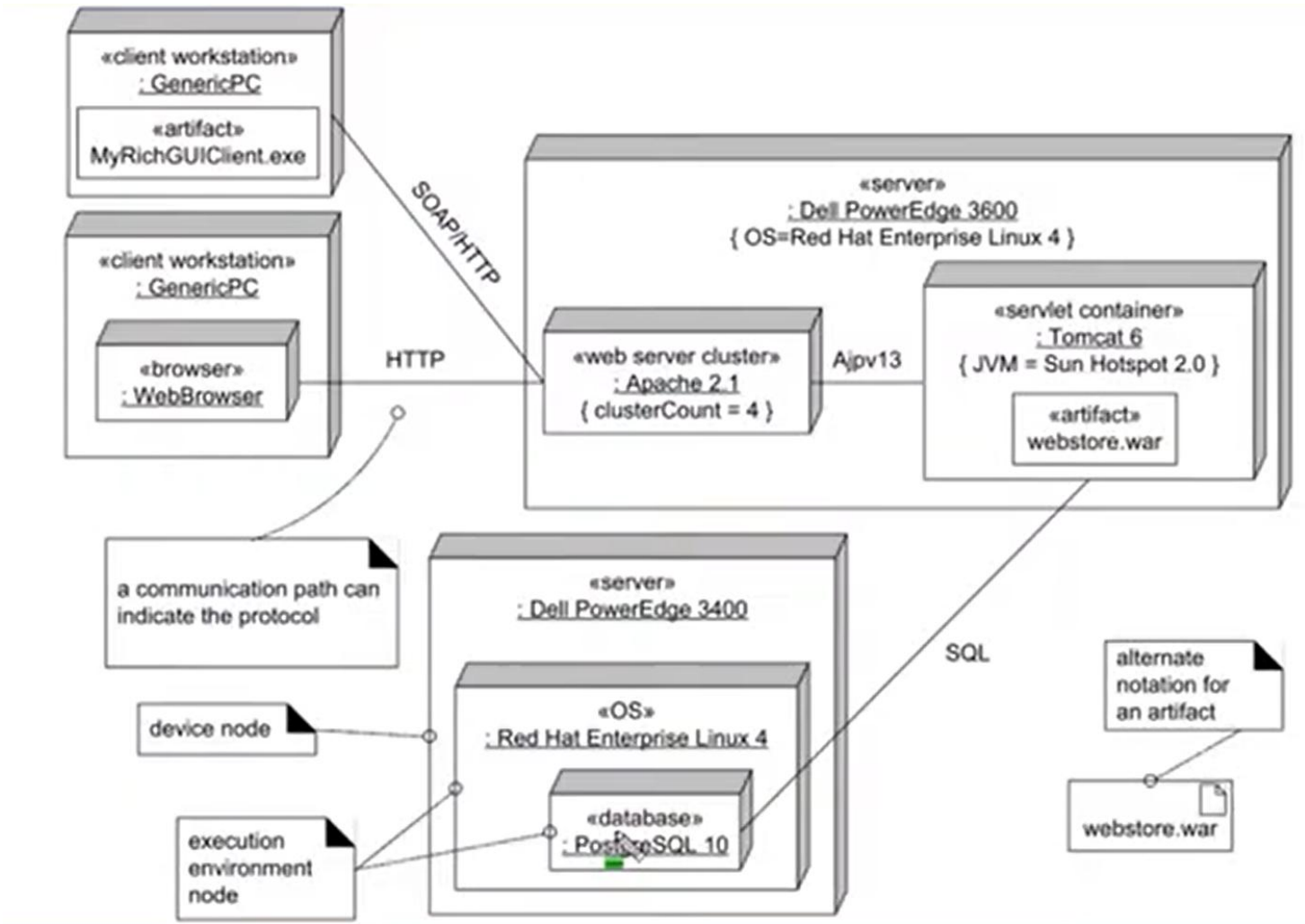
- Deployment diagrams model the physical architecture of a system, showing how software components are distributed across hardware.

They Show:
- Hardware devices (servers, phones)
- Software artifacts (JAR, WAR, EXE)
- Network connections
- Execution environments (OS, VM)

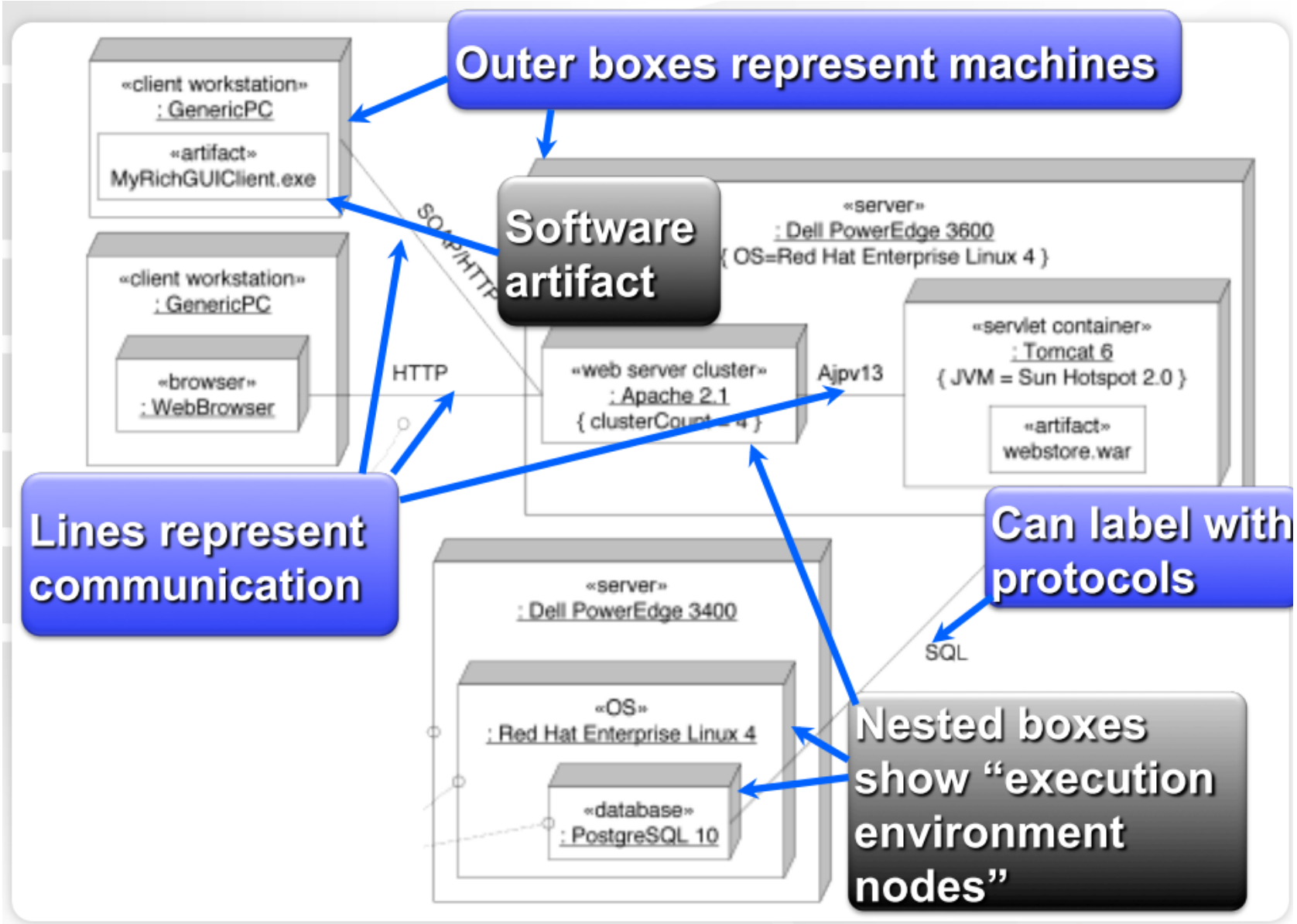
They Answer:
- Where does this run?
- How do systems communicate?
- What protocols are used?
- How is load distributed?

- Deployment diagrams also model the configuration of run-time processing nodes and the component instances and objects that live on them

Example

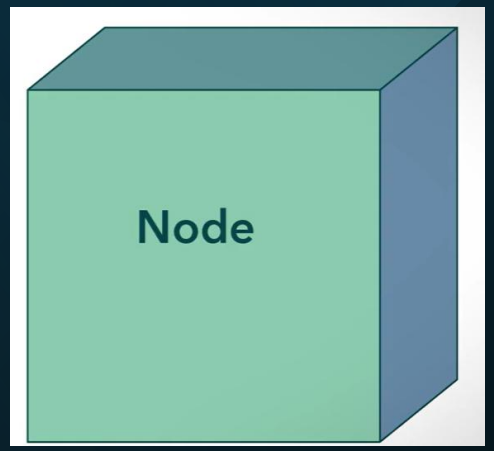

«client workstation»
: GenericPC
«artifact»
MyRichGUIClient.exe
«client workstation»
: GenericPC
«browser»
: WebBrowser
SOAP/HTTP
HTTP
«server»
: Dell PowerEdge 3600
{ OS=Red Hat Enterprise Linux 4 }
«web server cluster»
: Apache 2.1
{ clusterCount = 4 }
Ajpv13
«servlet container»
: Tomcat 6
{ JVM = Sun Hotspot 2.0 }
«artifact»
webstore.war
a communication path can indicate the protocol
«server»
: Dell PowerEdge 3400
SQL
alternate notation for an artifact
device node
«OS»
: Red Hat Enterprise Linux 4
«database»
: PostgreSQL 10
execution environment node
webstore.war

Annotated Example





Understanding Nodes



- Nodes are the building blocks of deployment diagrams. There are two types:
  - Device Node <<device>>
    - Physical Computing Resources with Processing Power and Memory
    - Examples: Web Server, Mobile Phone, Desktop PC, IoT Device
  - Execution Environment Node <<executionEnvironment>>
    - Software hosts and executes other software
    - Examples: JVM, Docker Container, Apache Tomcat, PostgreSQL, Web Browser

Key Point: Execution environments can be nested (Docker → OS → Physical Server)

- An Executable
- An Installer
- Audio Libraries
- Multimedia Assets



Artifacts and Manifestation

- Artifacts: Physical files that represent software components
  - JAR/WAR files: Java archives
  - EXE/DLL files: Executable programs
  - Docker Images: Container packages
  - Configuration files: XML, JSON, YAML

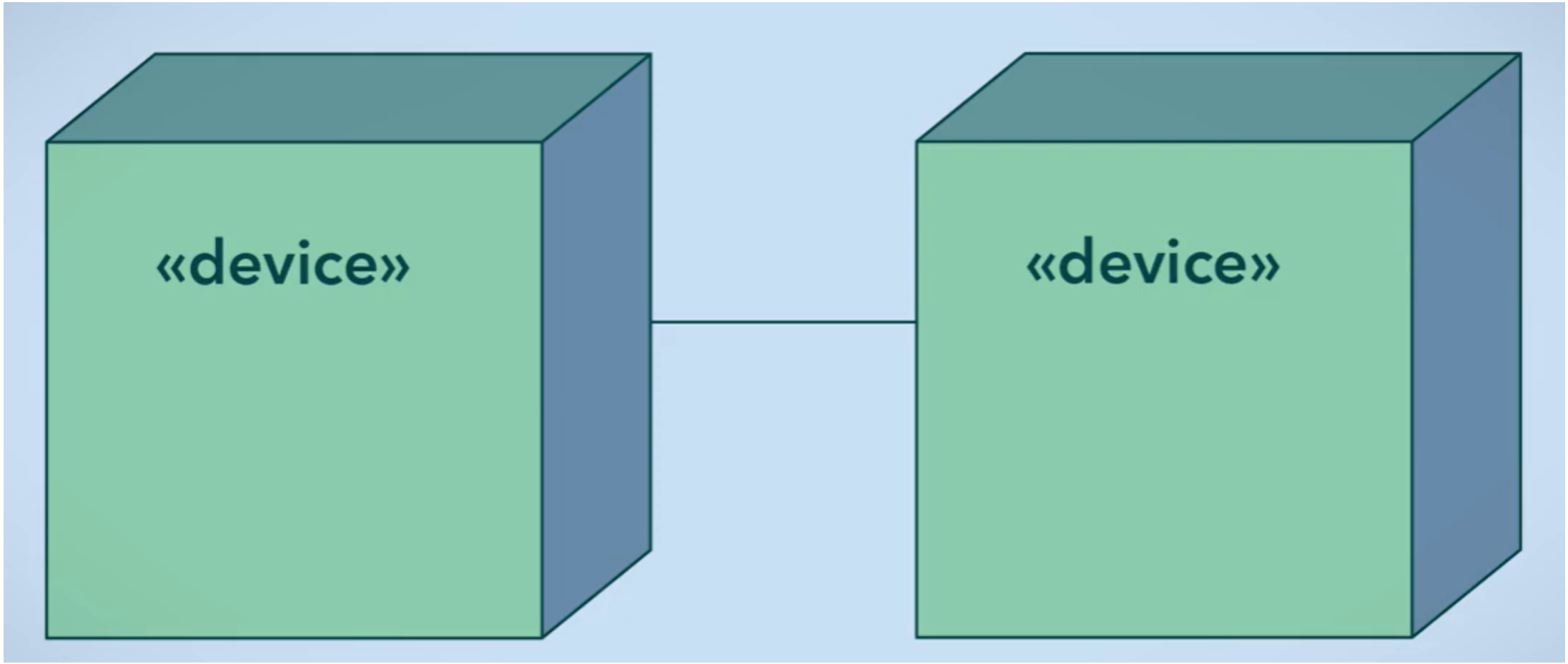
- Manifestation <<manifest>>: The relationship showing how components are implemented by artifacts

Component → «manifest» → Artifact

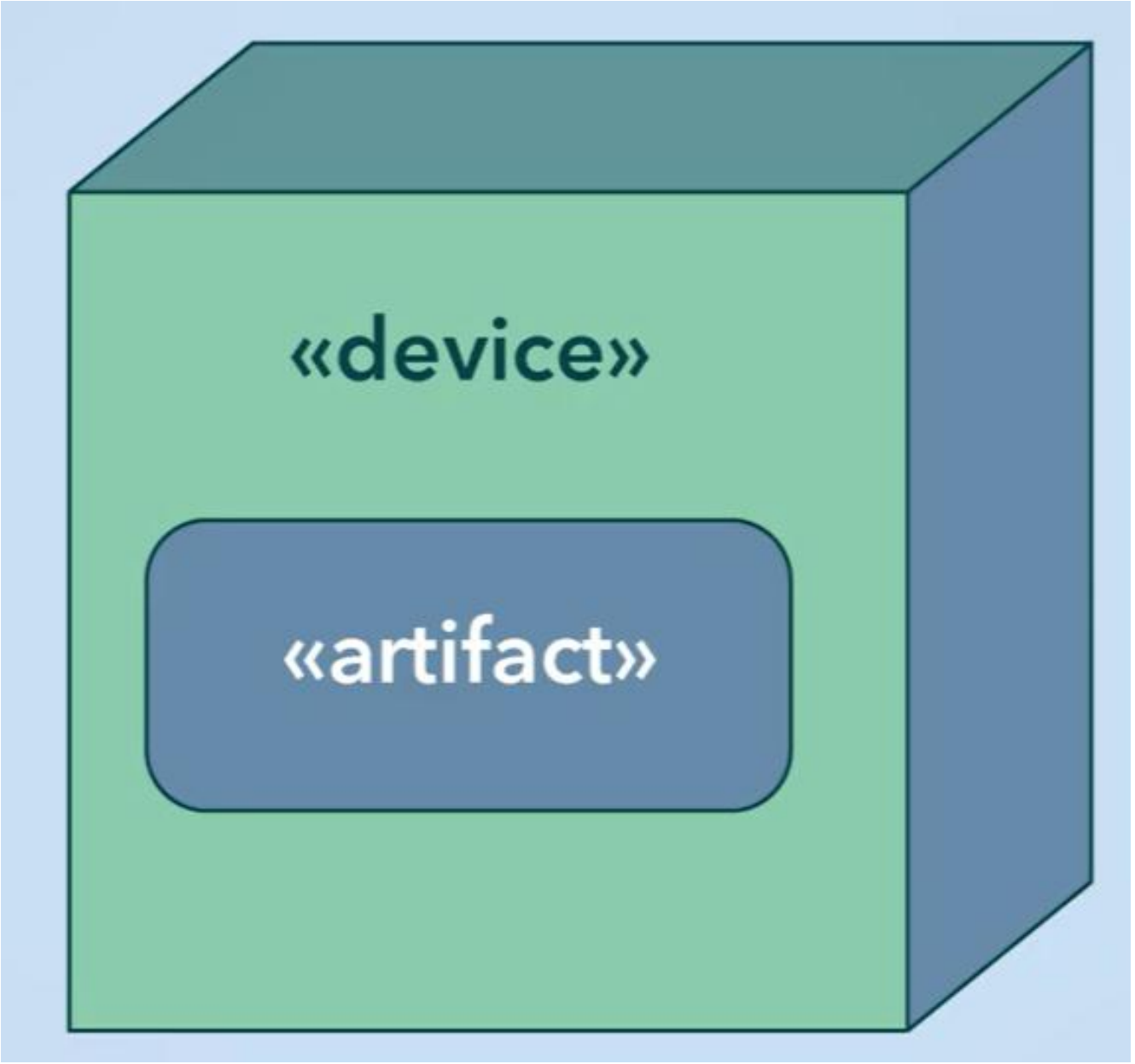
Example: PaymentService → «manifest» → payment-service.jar

Think: The component is the design; the artifact is the actual file you deploy

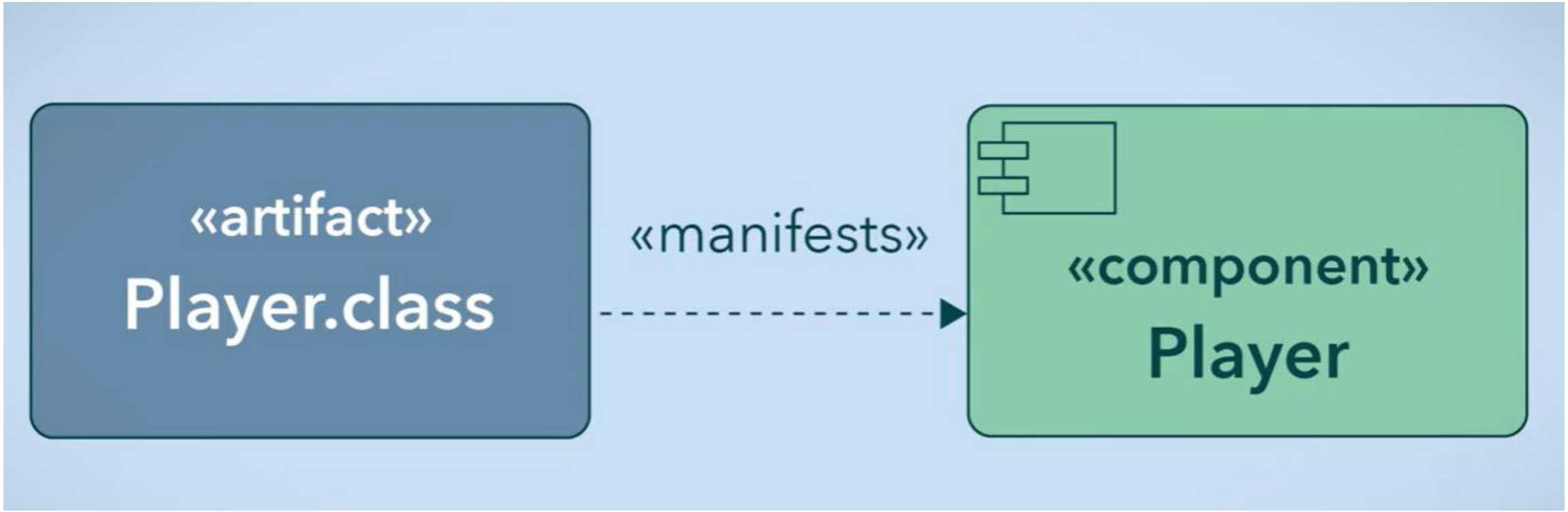
Relationship and Deployment Targets



Relationship and Deployment Targets



Relationship and Deployment Targets

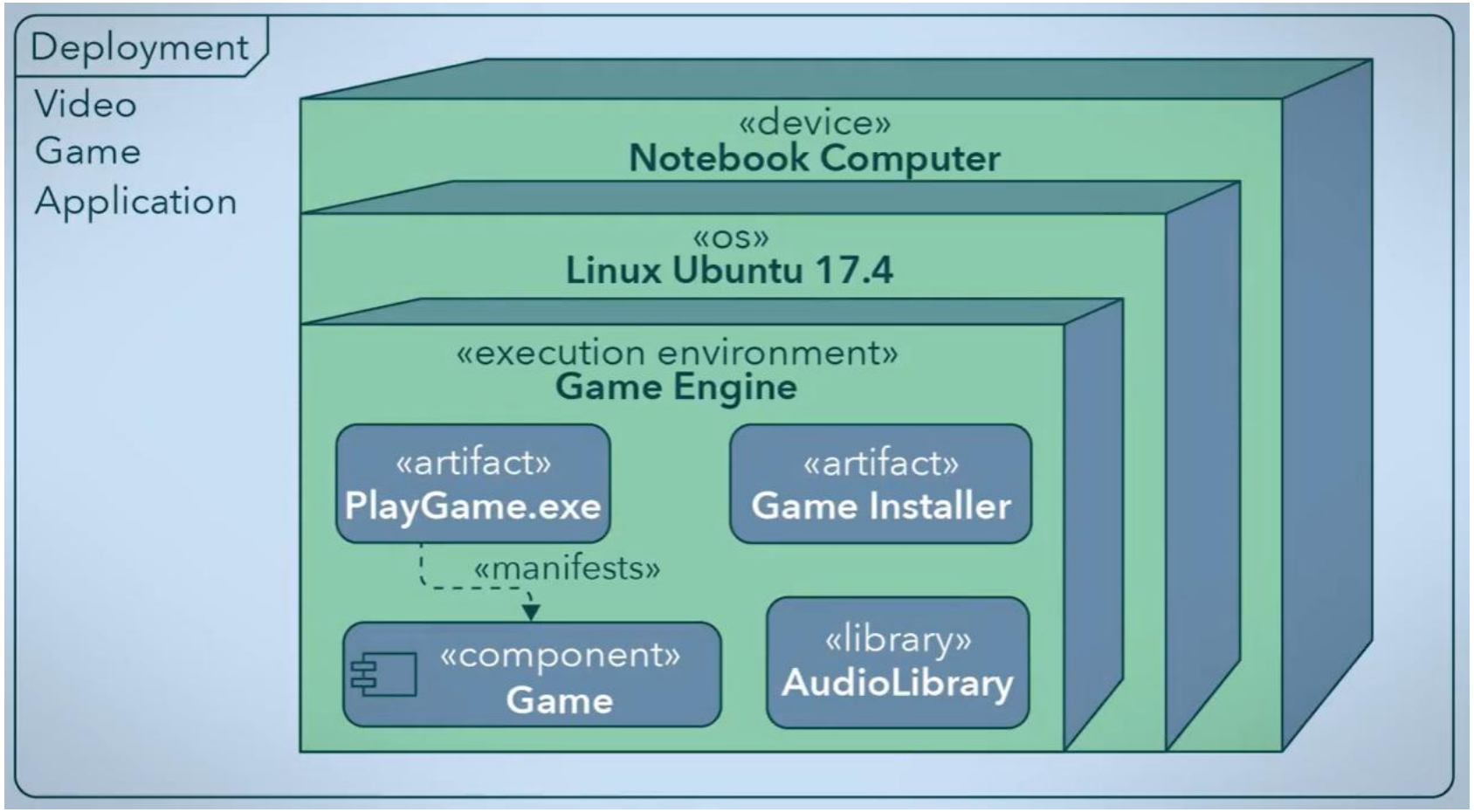


Deployment Diagrams

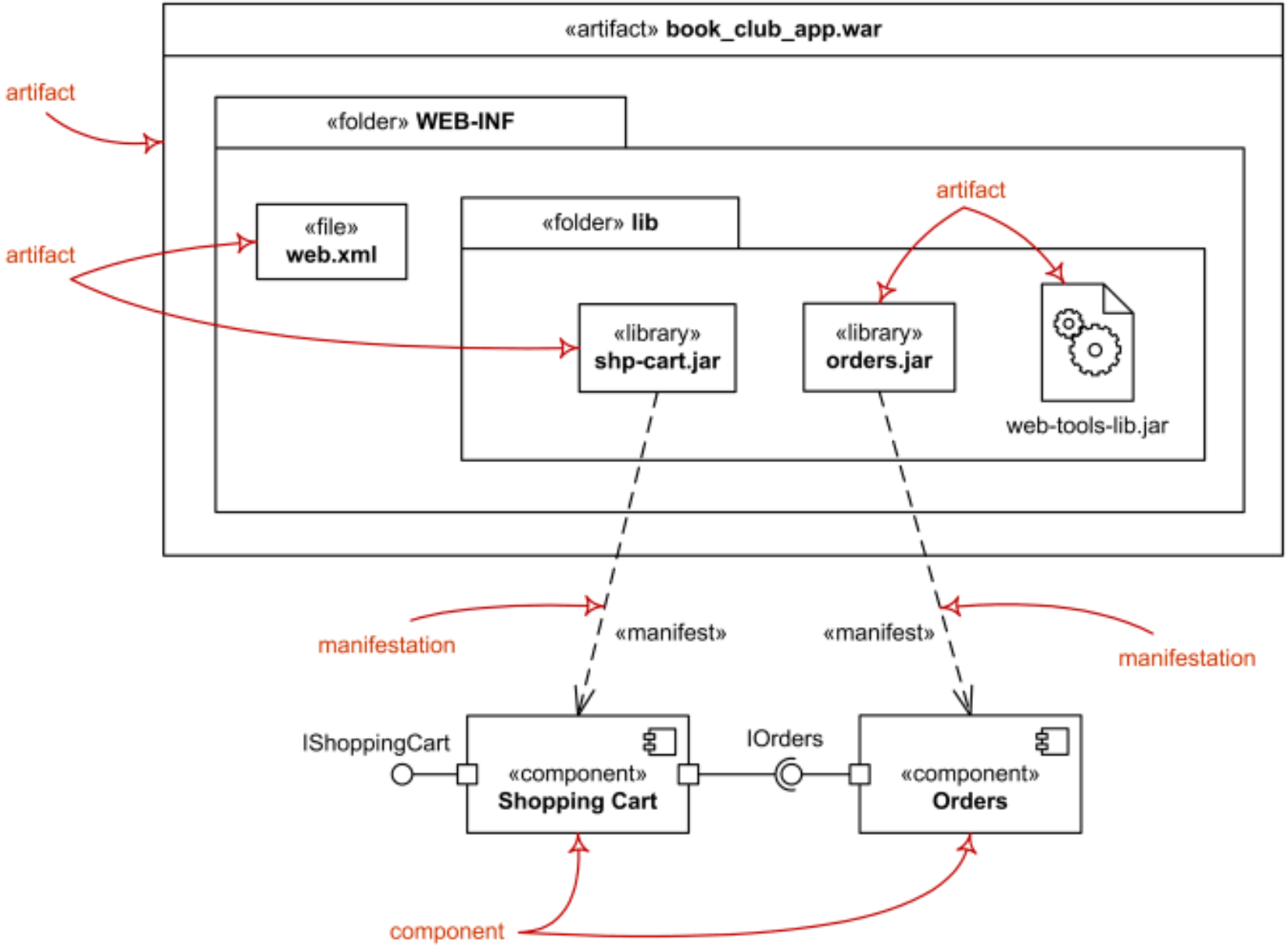
- As the UML specification suggests, many node types may show stereotypes, such as <<server>>, <<OS>>, <<database>>, or <<browser>>, but these are not official predefined UML stereotypes

- Note that a device node or EEN may contain another EEN
- For example, a virtual machine within an OS within a computer
- A particular EEN can be implied, or not shown, or indicated informally with a UML property string; for example, {OS = Linux}
- For example, there may not be value in showing the OS EEN as an explicit node.

Deployment diagram example

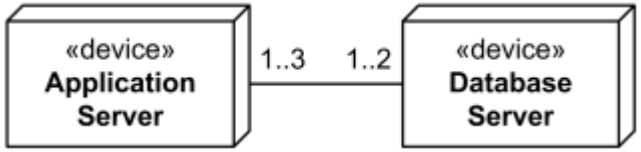


Manifestation of Components by Artifacts

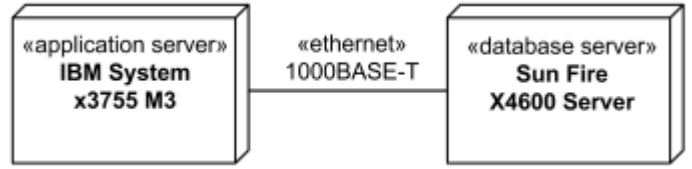


Communication Paths and Protocols

- Communication Paths
  - Represent Network Connections between nodes
  - Shown as lines connecting nodes, labeled with protocols or technologies



Communication path between several application servers and database servers.



Gigabit Ethernet as communication path between application and database servers.

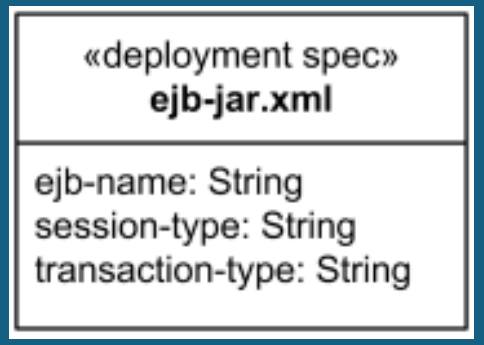
Common Protocols:
- HTTP/HTTPS - Web traffic
- TCP/IP - Network layer
- JDBC - Database connections
- WebSocket - Real-time data
- MQTT - IoT messaging

Network Types:
- Gigabit Ethernet
- WiFi / 5G
- VPN tunnels
- Message queues (RabbitMQ)
- REST APIs

Pro Tip: Always label communication paths with the protocol or bandwidth requirements

Deployment Specifications

- Configure artifacts that define HOW a component should be deployed and run



- **Examples in Practice:**

- docker-compose.yml
- Defines container configurations, networks, volumes

- application.properties
- Database URLs, port numbers, environment variables

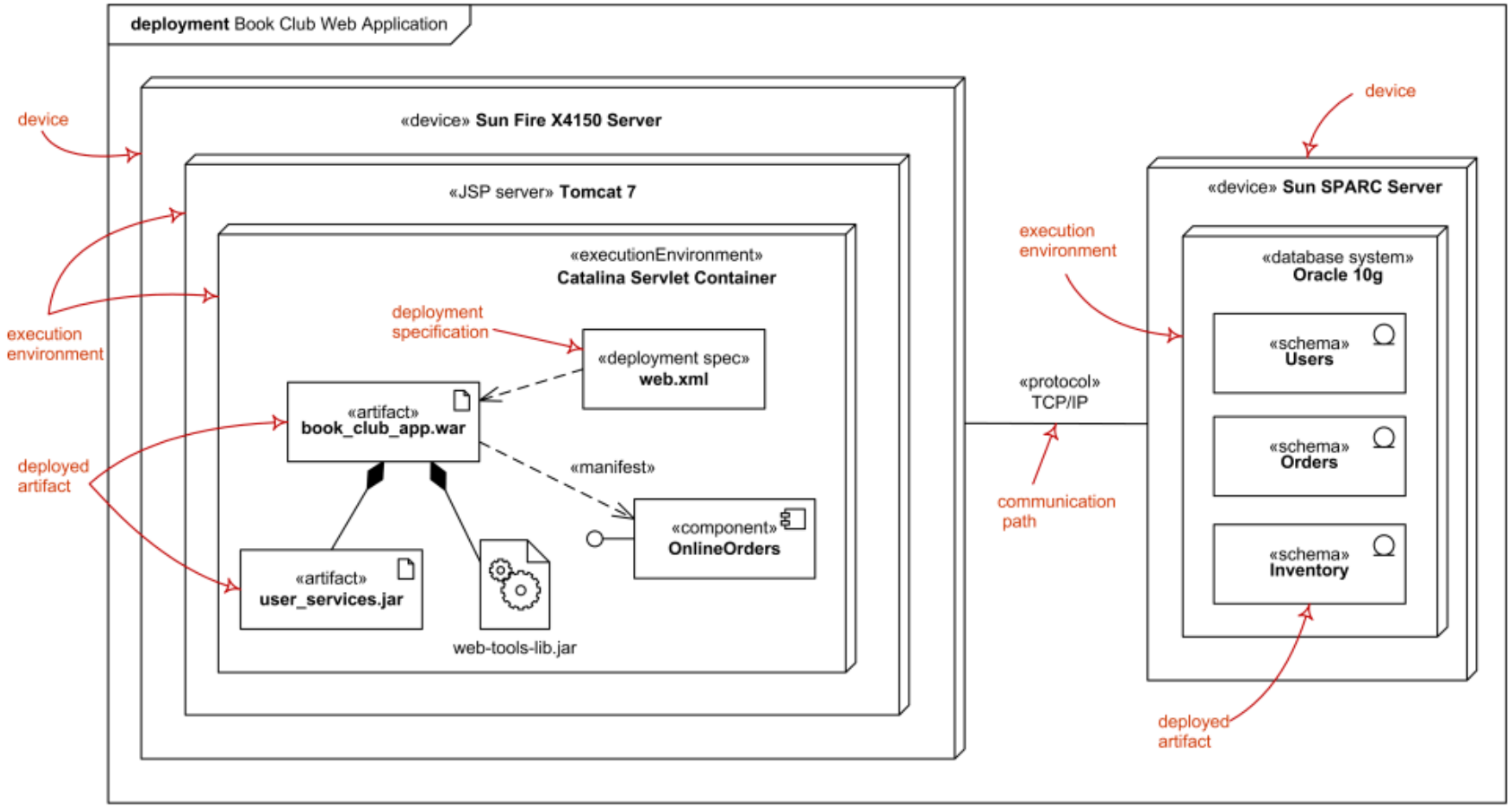
- kubernetes-deployment.yaml
- Replicas, resource limits, health checks

- web.xml / ejb-jar.xml
- Java EE servlet mappings, bean configurations

These specifications control memory limits, environment variables, startup parameters, etc.

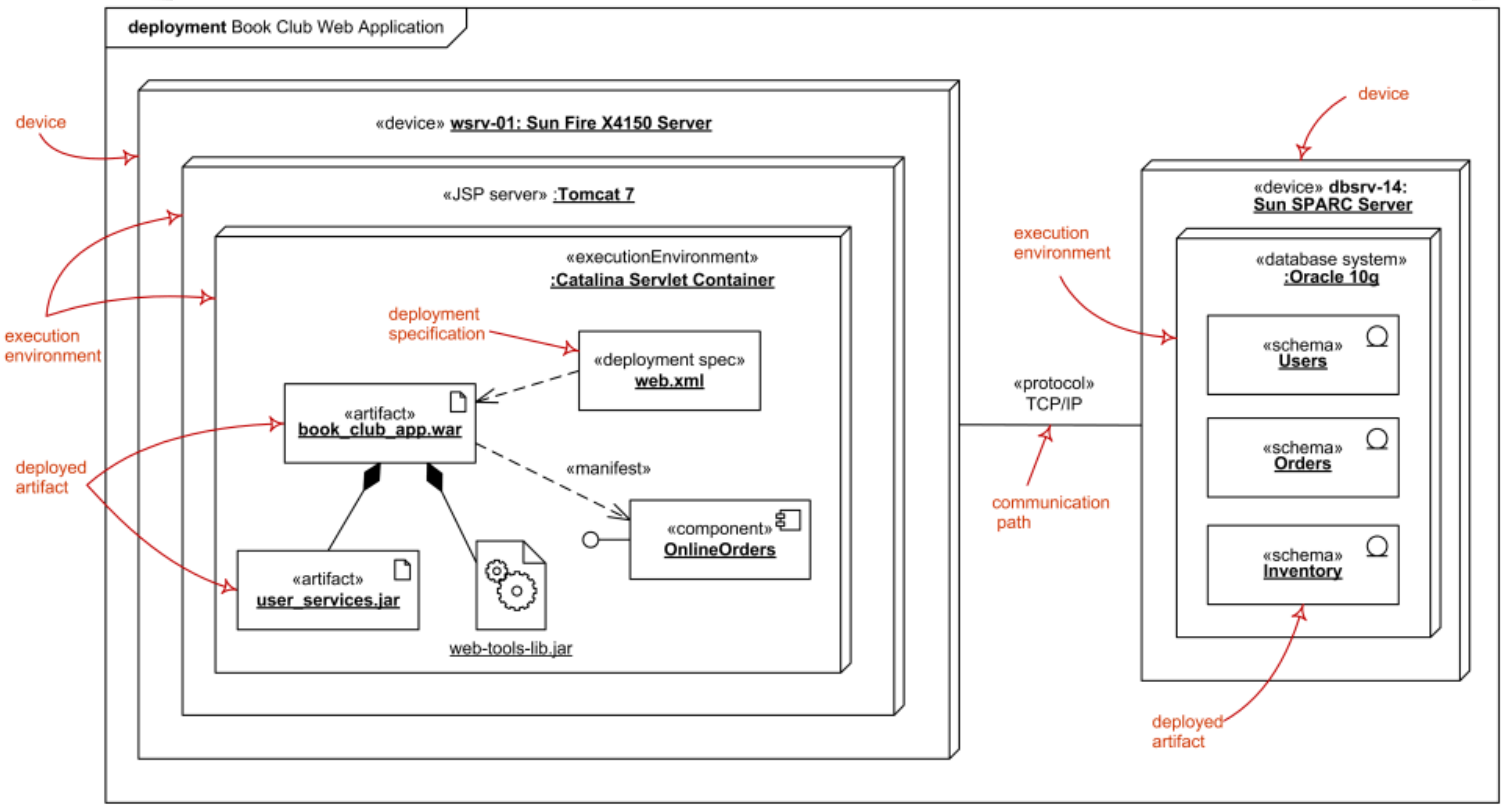
Specification Level Deployment Diagram

- A specification-level diagram gives an overview of artifacts and deployment targets, without referencing specific details like machine names. It focuses on a general overview of your deployment rather than specifics.

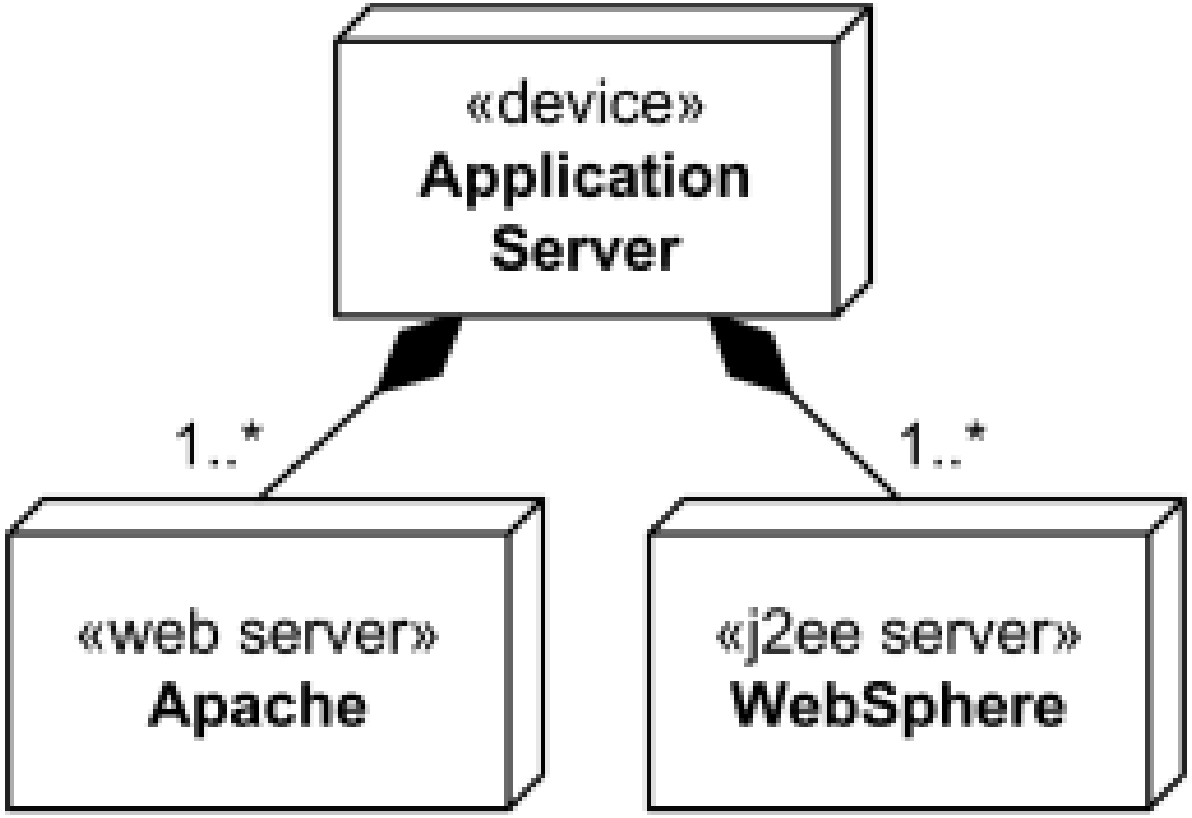


Instance Level Deployment Diagrams

- An instance level diagram is a much more specific approach, which can map a specific artifact to a specific deployment target. In particular, the instance level diagram can identify specific machine and hardware devices. Most commonly, this approach is used to highlight the differences in deployments among development staging and release builds.



Hierarchical nodes can be modeled using **composition** or by defining an **internal structure**.



Application server box runs several web servers and J2EE servers

Case Study: Deployment Architecture

Deployment Strategy:

Cloud-based microservices deployed on AWS using Docker containers

Frontend Tier
- EC2: Web Server
- Nginx + React

Application Tier
- EC2: App Servers (×3)
- Docker + JVM

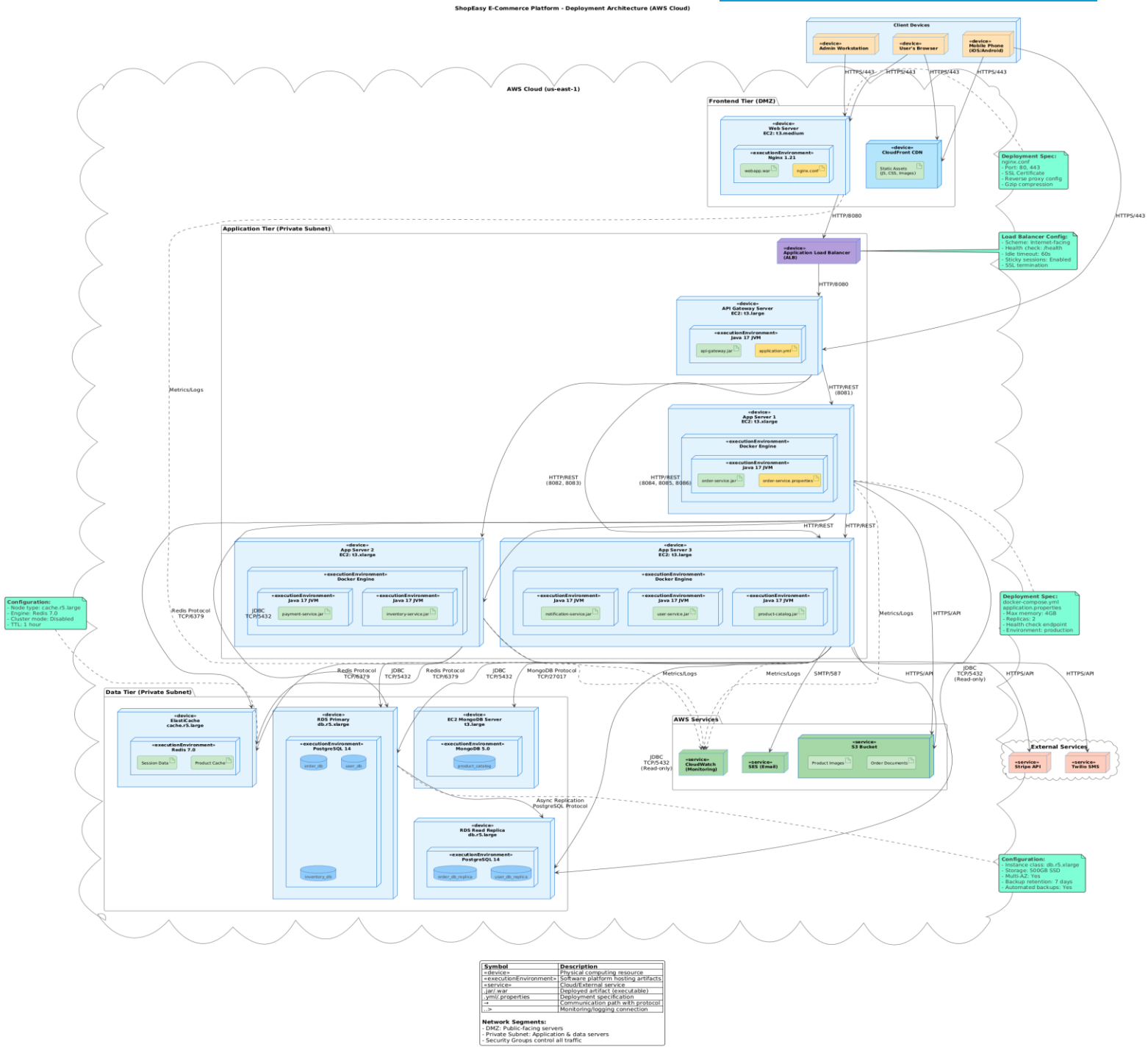
Data Tier
- RDS: PostgreSQL
- ElastiCache: Redis

Artifacts Deployed:
- webapp.war → Web Server
- order-service.jar → App Server 1
- payment-service.jar → App Server 2
- inventory-service.jar → App Server 2
- notification-service.jar → App Server 3

Communication Protocols:
- HTTPS/REST between frontend and backend
- JDBC for database connections
- Redis protocol for caching

Case Study: Deployment Diagram

High resolution image and PlantUML code is available on loop



Component vs Deployment Diagrams

Component Diagram

Focus: Logical organization

Shows: Software structure

Elements: Components, interfaces

Question: "What are the parts?"

Level: Design/Development

Changes: As code evolves

Deployment Diagram

Focus: Physical distribution

Shows: Hardware topology

Elements: Nodes, artifacts

Question: "Where does it run?"

Level: Operations/DevOps

Changes: With infrastructure

Relationship:

Component diagrams show WHAT you're building (logical architecture). Deployment diagrams show WHERE and HOW you're running it (physical architecture). You need both for a complete system understanding!

Real-world Example: Netflix Architecture

This is the power of proper component and deployment architecture!

Netflix Microservices

Netflix operates over 700 microservices, each handling specific functionality

Components:

- User Service (authentication)
- Recommendation Engine
- Video Transcoding Service
- Payment Service
- API Gateway (Zuul)

Deployment:

- AWS Cloud (EC2, S3, CloudFront)
- Cassandra databases (replicated)
- Thousands of server instances
- Global CDN for content delivery
- Auto-scaling groups

Key Takeaways:

- Each service is independently deployable and scalable
- Services communicate via REST APIs and message queues
- Deployment spans multiple AWS regions for reliability
- Chaos Engineering (deliberately breaking things) to test resilience

Additional Resources

- Applying UML and Patterns by Craig Larman (Chapters on Architecture)
- Software Architecture in Practice by Bass, Clements, Kazman
- Building Microservices by Sam Newman
- Designing Data-Intensive Applications by Martin Kleppmann

Additional Resources

- Some good videos
- https://www.youtube.com/watch?v=IzEzX5HW_CU
- https://www.youtube.com/watch?v=ipKJwnPsst8
- https://www.youtube.com/watch?v=A6XZwa3yv6s
- https://www.youtube.com/watch?v=XU6OMqf94u0